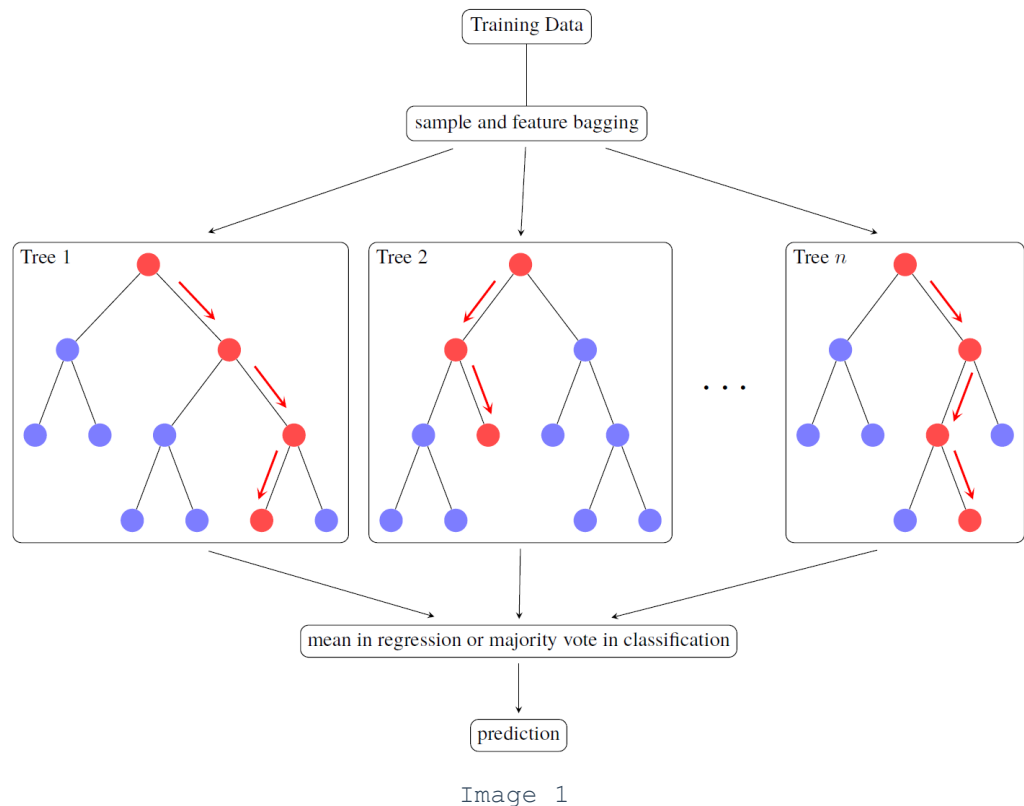


随机森林



1. 决策树

学习随机森林之前，一定要先理解决策树，因为随机森林就是由这一颗颗决策树构成的。随机森林是基于决策树的最强大算法。

决策树是一种预测模型，它是属于有监督学习的一种算法。它通过一系列基于特征的分裂规则，将特征空间递归地划分为不同的区域，最终为每个区域分配一个预测结果。

对于分类任务，预测结果是类别标签；对于回归任务，预测结果是一个连续值。

决策树的构建过程依赖于节点**纯度**的度量，即节点内样本类别的一致性程度。一个理想的分裂应该使得分裂后的子节点比父节点具有更高的纯度。

定义1.1 (节点纯度与分裂准则) 为了量化节点的纯度，信息熵和基尼不纯度是常用的分裂准则：

- 信息熵:** 熵是衡量随机变量不确定性的度量。对于一个包含 n 个类别的数据集，其信息熵定义为：

$$Ent(D) = - \sum_{i=1}^n p_i \log_2 p_i$$

其中 p_i 是数据集中第 i 类样本所占的比例。熵值越大，表示数据集的不确定性越高，纯度越低。决策树在分裂时，会选择能够最大化信息增益（即分裂前后熵值下降最多）的特征进行分裂。

信息增益的计算：

$$Gain(D, a) = Ent(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} Ent(D^v)$$

- **基尼不纯度**: 这是另一种衡量纯度的指标，定义为从数据集中随机抽取两个样本，其类别标记不一致的概率。对于一个包含 K 个类别的数据集，其基尼不纯度为：

$$Gini(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

其中 p_k 是第 k 类样本的比例。基尼不纯度的值越小，表示数据集的纯度越高。

一个属性的信息增益或 $Gini$ 指数越大，表明属性对样本的熵减少的能力更强，这个属性使得数据由不确定性变成确定性的能力越强。

实践中，信息熵和基尼不纯度的效果相近，但后者计算更快，因此是 `scikit-learn` 等库的默认选择，这对高效构建随机森林很重要。

定理 1.1 (递归划分) 决策树的构建是一个自顶向下的贪心过程，称为递归划分。算法从根节点开始，在每个节点上选择能最大化子节点纯度的最优特征进行分裂，并对子节点重复这个过程，直到满足停止条件。

2. 随机森林

2.1 随机森林的定义

定义 2.1 (随机森林) 随机森林是一个包含多棵决策树的集成学习器，其中每棵树的构建过程都引入了双重随机性：

1. **样本随机化**：每棵树都在一个通过自助采样（有放回抽样）得到的自助样本集上进行训练。
2. **特征随机化**：在每个节点的最佳分裂选择过程中，不再是考虑所有可用特征，而是从全部特征中随机抽取一个子集，然后从这个子集中选择最优分裂特征。

双重随机性确保了各决策树间的差异性（即去相关），这正是随机森林能有效降低方差、提升模型稳定性的关键。

2.2 算法详解

随机森林的训练过程的三个核心步骤：

1. **自助采样**: 假设原始训练集有 N 个样本。对于森林中的每一棵树（共 `n_estimators` 棵），都从原始训练集中进行有放回的随机抽样，抽取 N 个样本，形成该树的训练集。
2. **带特征子集的决策树生长**: 对每一个自助样本集，生长一棵决策树。与标准决策树不同的是，在每个节点进行分裂时：
 - 从全部 M 个特征中，随机选择一个包含 m 个特征的子集，其中 m 是一个远小于 M 的超参数，记为 `max_features`。

在 `scikit-learn` 中，对于分类问题，`m` 的默认值通常是 $\sqrt{M}$ ；对于回归问题，默认值是 $M/3$ 或 M 。

根据 `scikit-learn` 的官方文档，从 1.1 版本开始，回归问题的默认值已经从 M 改为了 $\sqrt{M}$ 。

- 算法仅在这个 m 个特征子集中寻找最佳的分裂特征和分裂点。

- 决策树持续生长，直到达到某个停止条件（如达到最大深度、叶节点样本数小于阈值等）。为了使每棵树具有低偏差，随机森林中的树通常会生长到最大深度，不进行剪枝。
3. **预测聚合**: 当需要对一个新的数据点进行预测时，该数据点会被输入到森林中的每一棵树中，得到 `n_estimators` 个独立的预测结果。
- **分类问题**: 最终的预测结果由所有树的预测结果通过**多数投票**决定。**回归问题**: 最终的预测结果是所有树预测值的**算术平均值**。

`max_features` 直接决定了森林中个体树的强度与树之间相关性的平衡。

当 `max_features` 值较大（例如等于总特征数 M ）时，随机森林退化为标准的 Bagging 算法，每棵树都有机会选择强预测特征，因此个体树的强度（低偏见）较高，但树之间的相关性也较强，导致集成后方差降低有限。

当 `max_features` 值较小时，每棵树在分裂时受到更多限制，可能无法选择到最优特征，导致个体树的强度变弱（偏见增大），但由于特征选择的随机性，树之间的相关性大大降低，使得集成后的方差能够得到更有效的控制。

我们一般用 $\sqrt{M}$ ，是在个体树强度和树间相关性之间寻求的一个有效平衡点。

2.3 袋外误差

随机森林的一个优雅特性是它内置提供了一种无需额外计算成本的交叉验证方法，即袋外误差估计。

由于自助采样的特性，对于每个自助样本集，平均约有 $1 - (1 - 1/N)^N \approx 1 - 1/e \approx 63.2\%$ 的原始样本被抽中。这意味着，对于森林中的每一棵树，大约有 36.8 的原始训练样本没有被用于其训练过程。这些未被使用的样本被称为该树的袋外样本。

定义 2.2 (袋外误差) 袋外误差是利用这些袋外样本来评估模型泛化能力的一种方法。其计算过程如下：

1. 对于训练集中的每一个样本 i ，找出所有在构建过程中未使用到样本 i 的树的集合。
2. 让这个树的集合对样本 i 进行预测。
3. 将这个预测结果与样本 i 的真实标签进行比较，计算出误差。
4. 对所有训练样本重复这个过程，最终得到的平均误差即为袋外误差。

随机森林的泛化误差会随着树的数量增加而收敛到一个极限值。这意味着，与许多其他机器学习模型不同，随机森林不会因为增加更多的树 (`n_estimators`) 而产生过拟合（这个结论是在单棵决策树不剪枝（或轻微剪枝）的典型设置下成立的。增加树的数量主要是降低模型的方差，而不会增加偏差，因此总误差趋于稳定。）。所以要树的数量足够多，袋外误差就会稳定下来。所以在实践中，可以将 `n_estimators` 设置为一个足够大的值，直到模型性能到瓶颈，而不用担心过拟合问题，唯一的代价是计算时间的增加。

3. 代码实现

3.1 准备

先配置Python环境，导入必要的库。

再准备数据，加载数据集，并将其划分为训练集和测试集（这里使用鸢尾花数据集来做分类任务的演示）。

```

1  # 导入基础库
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import seaborn as sns
6
7  # 从 scikit-learn 中导入所需模块
8  from sklearn.datasets import load_iris
9  from sklearn.model_selection import train_test_split
10 from sklearn.ensemble import RandomForestClassifier
11 from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report
12
13 # 设置 Matplotlib 字体以支持中文显示
14 plt.rcParams['font.sans-serif'] = 'Microsoft YaHei'
15 plt.rcParams['axes.unicode_minus'] = False
16
17 # 加载鸢尾花数据集
18 iris = load_iris()
19 X = pd.DataFrame(iris.data, columns=iris.feature_names)
20 y = pd.Series(iris.target, name='species')
21
22 # 打印数据概览
23 print("特征数据前5行:")
24 print(X.head())
25 print("\n目标数据概览:")
26 print(y.value_counts())
27
28 # 将数据划分为训练集和测试集
29 # random_state=42 可选的, 保证实验的可复现性
30 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42, stratify=y)
31
32 print(f"\n训练集大小: {X_train.shape} 样本")
33 print(f"\n测试集大小: {X_test.shape} 样本")

```

Fence 1

3.2 训练基准模型

建立一个使用默认参数的基准模型作为性能参考点。

```

1  # 1. 使用默认参数初始化随机森林分类器
2  # 可选, 设置 random_state=42
3  rf_baseline = RandomForestClassifier(random_state=42)
4
5  # 2. 在训练数据上拟合模型
6  rf_baseline.fit(X_train, y_train)
7
8  print("基准随机森林模型训练完成。")
9  print(f"模型使用的树的数量 (n_estimators): {rf_baseline.n_estimators}")
10 print(f"模型使用的分裂准则 (criterion): {rf_baseline.criterion}")

```

Fence 2

3.3 预测和评估

使用测试数据进行预测，并评估。

```
1  # 1. 使用训练好的基准模型对测试集进行预测
2  y_pred_baseline = rf_baseline.predict(X_test)
3
4  # 2. 计算并打印准确率
5  accuracy_baseline = accuracy_score(y_test, y_pred_baseline)
6  print(f"基准模型准确率: {accuracy_baseline:.4f}")
7
8  # 3. 打印详细的分类报告
9  print("\n基准模型分类报告:")
10 print(classification_report(y_test, y_pred_baseline,
11                             target_names=iris.target_names))
12
13 # 4. 生成并可视化混淆矩阵
14 cm_baseline = confusion_matrix(y_test, y_pred_baseline)
15
16 plt.figure(figsize=(8, 6))
17 sns.heatmap(cm_baseline, annot=True, fmt='d', cmap='Blues',
18             xticklabels=iris.target_names, yticklabels=iris.target_names)
19 plt.title('基准模型混淆矩阵')
20 plt.ylabel('真实类别')
21 plt.xlabel('预测类别')
22 plt.show()
```

Fence 3

图片输出

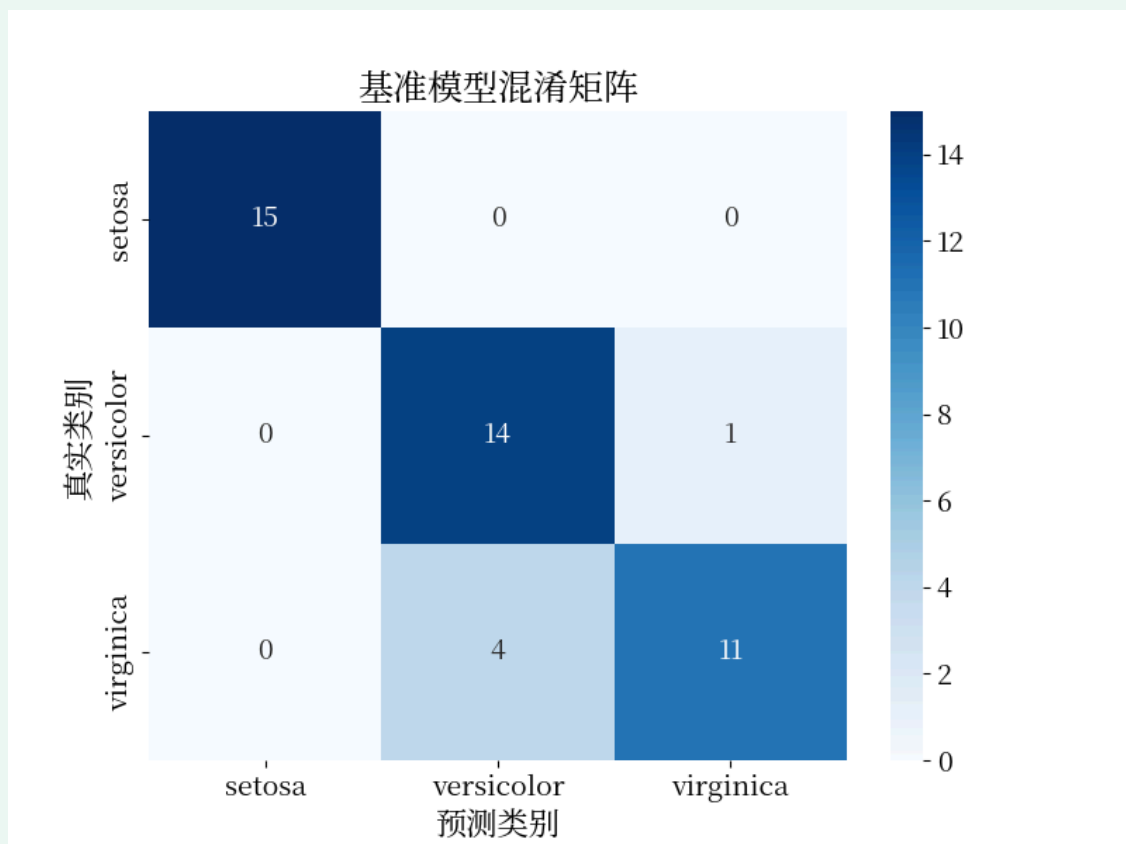


Image 2

以上是一个可工作的随机森林模型，接下来还需要进行调优。

3.4 超参数调优

在训练前的设置叫**超参数**，需要我们手动指定。优化后可以提升模型泛化能力。对于随机森林，最重要的几个超参数包括：

- `n_estimators`：森林中决策树的数量。
- `max_depth`：每棵决策树的最大深度。
- `max_features`：每个节点分裂时考虑的特征数量。
- `min_samples_leaf`：一个叶节点必须具有的最小样本数量。

定义 4.1 (网格搜索交叉验证) 它是一种穷举式的超参数搜索方法。首先定义要搜索的超参数网格，然后对所有组合遍历。对于每一种组合，都使用 K-折交叉验证来评估性能，最终选出平均性能最优的。

```
1     from sklearn.model_selection import GridSearchCV
2
3     # 1. 定义要搜索的超参数网格
4     # 这个字典的键是超参数名称，值是待测试的候选列表
5     param_grid = {
6         'n_estimators': [50, 100, 200],
7         'max_depth': [None, 10, 20],
8         'max_features': ['sqrt', 'log2', None],
9         'min_samples_leaf': [1, 2, 4],
10        'criterion': ['gini', 'entropy']
11    }
12
13    # 2. 初始化网格搜索模型
14    # estimator: 待调优的模型
15    # param_grid: 超参数网格
16    # cv=5: 使用5折交叉验证
17    # n_jobs=-1: 使用所有可用的CPU核心并行计算，加快搜索速度
18    # verbose=2: 打印详细的搜索过程信息
19    grid_search = GridSearchCV(estimator=RandomForestClassifier(random_state=42),
20                              param_grid=param_grid,
21                              cv=5,
22                              n_jobs=-1,
23                              verbose=2)
24
25    # 3. 在训练数据上执行网格搜索
26    print("开始进行超参数网格搜索...")
27    grid_search.fit(X_train, y_train)
28    print("网格搜索完成。")
29
30    # 4. 打印找到的最佳超参数组合
31    print(f"\n最佳超参数组合: {grid_search.best_params_}")
32
33    # 5. 使用最佳超参数的模型进行预测和评估
34    best_rf = grid_search.best_estimator_
35    y_pred_tuned = best_rf.predict(X_test)
36
37    # 评估调优后的模型
38    accuracy_tuned = accuracy_score(y_test, y_pred_tuned)
39    print(f"\n调优后模型准确率: {accuracy_tuned:.4f}")
40    print("\n调优后模型分类报告:")
```

```
41 print(classification_report(y_test, y_pred_tuned,
                             target_names=iris.target_names))
```

Fence 4

超参数调优的结果本身也是很有价值的，可以帮助我们更深入理解问题。比如，如果网格搜索最终选择了一个较小的 `max_depth`，可能暗示数据集中的关系相对简单，不需要复杂的特征交互就能很好地建模，我们就可以避免使用过于复杂的模型。反之则说明数据中可能存在高阶的非线性关系。

表 4.1: 模型性能调优前后对比

为了清晰地展示超参数调优带来的价值，将基准模型与调优后模型的性能指标进行对比是至关重要的。

评价指标	基准模型	调优后模型	性能提升
准确率	0.9333	0.9556	+2.39%
精确率	0.94	0.96	+2.13%
召回率	0.94	0.96	+2.13%
F1分数	0.94	0.96	+2.13%

Table 1

注：上表中的数值是基于本次示例运行的典型结果，实际运行结果可能因 `random_state` 或库版本不同而有微小差异。

3.5 特征重要性分析

特征重要性分析能够量化每个输入特征对模型预测的贡献程度。

定义 4.2 (平均不纯度减少) 也称为基尼重要性，`scikit-learn` 中的默认方法，计算速度快，但会偏向于高估拥有许多唯一值的特征（高基数特征）的重要性。

定义 4.3 (排列重要性) 一种更可靠、模型无关的方法。它通过打乱某一特征的数值，观察模型性能下降的幅度来评估其重要性。这个方法结果更可靠，但计算成本更高。

初步探索时可以用第一个方法，速度快，但在最终报告或做出关键决策（如特征选择）时，最好用排列重要性。

计算并可视化特征重要性（平均不纯度减少）：

```
1  # 1. 从调优后的最佳模型中获取特征重要性
2  importances = best_rf.feature_importances_
3  feature_names = X.columns
4
5  # 2. 创建一个包含特征名称和其重要性分数的 Pandas Series
6  forest_importances = pd.Series(importances,
7                                  index=feature_names).sort_values(ascending=False)
7
8  # 3. 绘制条形图进行可视化
9  plt.figure(figsize=(10, 6))
10 sns.barplot(x=forest_importances.values, y=forest_importances.index)
```

```

11 plt.title('基于平均不纯度减少 (MDI) 的特征重要性')
12 plt.xlabel('重要性分数')
13 plt.ylabel('特征')
14 plt.tight_layout()
15 plt.show()
16
17 print("各特征的重要性分数:")
18 print(forest_importances)

```

Fence 5

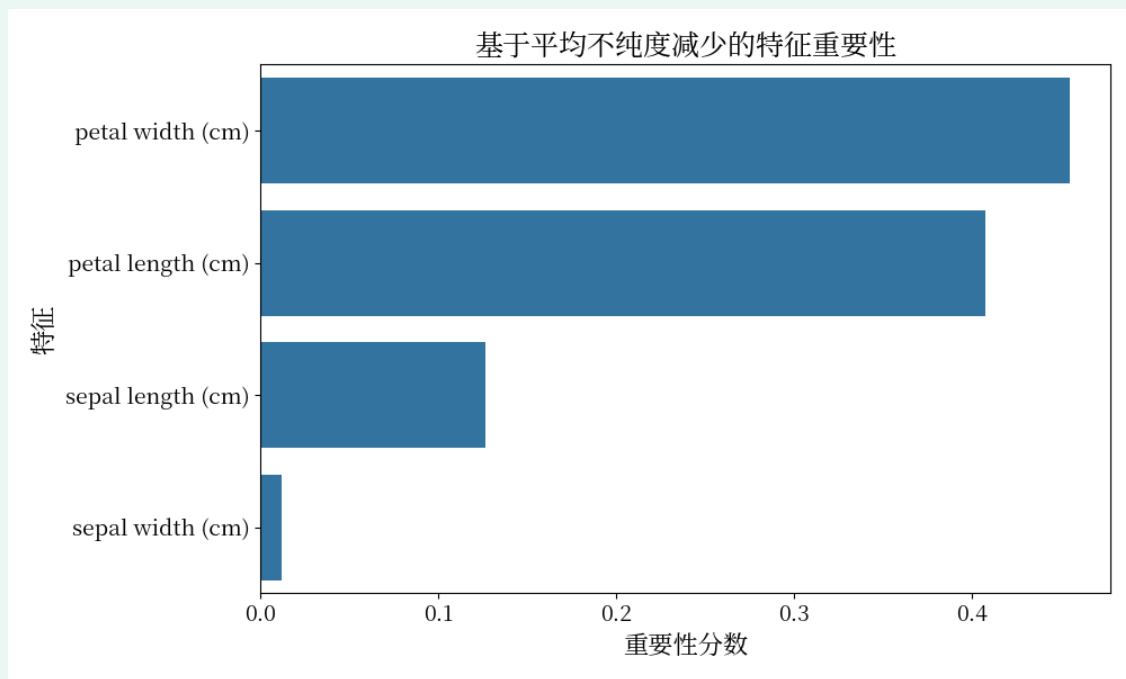


Image 3

从输出的图表以看出，在鸢尾花数据集中，“花瓣宽度”和“花瓣长度”是最具预测能力的特征，这与植物学领域的常识相符。